

VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Employee Payroll Management System

Object Oriented Programming through Java Laboratory

Submitted by

Modepalli Vivek 25881A05EQ

Course Facilitator

Mr. D. Ganesh, Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Contents

Abstract **2**

1 Introduction **2**

1.1 Audit note: what was supplied, and what was built 2

2 Literature Review **3**

3 Objectives **3**

4 Methodology **4**

4.1 Architectural Diagram 4

4.2 Hardware and Software Requirements 4

4.3 Working Principle 5

4.4 Software Implementation 6

5 Results and Discussion **7**

5.1 Limitations and Future Enhancements 8

6 Mapping POs and SDGs **9**

6.1 Program Outcomes (POs) Mapping 9

6.2 Sustainable Development Goals (SDGs) Mapping 9

7 Conclusion **9**

Bibliography **10**

Abstract

In many small organisations, employee salary calculation is still done manually or with the help of spreadsheets. Manual payroll work is slow, and a small mistake in an allowance or a deduction affects the salary of an employee directly. This project presents an Employee Payroll Management System developed in Java. The system is a console-based, menu-driven application that stores employee records in an `ArrayList` and provides operations to add, display, search, update and delete employees. For every employee, the system calculates the House Rent Allowance (HRA), Dearness Allowance (DA), Provident Fund (PF) and professional tax from the basic salary using fixed rules, and generates a complete payslip showing the gross salary and the net salary. It also produces a department-wise summary of the total payroll. The project is designed using object-oriented concepts: the `Person` class acts as a base class, the `Employee` class inherits from it and encapsulates the salary calculation, and the main class `EmployeePayrollSystem` manages the menu and the employee collection. Input validation and exception handling are used so that wrong inputs, such as a non-numeric salary or an invalid menu choice, do not stop the program. The system was tested with sample employee records, and the calculated payslips were verified by hand calculation.

Keywords: Employee payroll, salary calculation, payslip, Java, object-oriented programming, `ArrayList`, exception handling.

1 Introduction

Payroll is the process by which an organisation pays salaries to its employees. For every employee, the payroll calculation takes the basic salary and adds allowances such as House Rent Allowance (HRA) and Dearness Allowance (DA), and then subtracts deductions such as the Provident Fund (PF) contribution and professional tax. The final amount paid to the employee is the net salary. When this calculation is done manually for many employees, it takes a long time and calculation mistakes are common.

The Employee Payroll Management System developed in this project automates this process for a small organisation. It maintains a record of every employee, containing the employee ID, name, phone number, department, designation and basic salary. Using a simple menu, the user can add a new employee, view all employees, search for an employee by ID or by name, update an employee's details, delete a record, generate a printed payslip for one employee, and view the total payroll department by department. Because the salary rules are written once inside the `Employee` class, the same calculation is applied correctly and uniformly to every employee.

This project also serves as a practical application of object-oriented programming in Java. It uses classes and objects, inheritance, encapsulation, collections (`ArrayList`) and exception handling, which are the core topics of the Object Oriented Programming through Java Laboratory course.

1.1 Audit note: what was supplied, and what was built

The material supplied for this work was the earlier course-end report itself (as a PDF), not a ZIP containing runnable source code. That report described the system and printed part of the `Employee` class, but it did not contain a complete, runnable program: the `Person` class and the menu class were described but not listed in full, and the printed code excerpt was garbled in places in the PDF text (for example, the gross-salary line as extracted from the PDF appeared incomplete). This report therefore documents a complete implementation rebuilt from the supplied description, preserving its stated purpose, class design, menu and salary rules, and

verifying the result by compilation and testing rather than by repeating the supplied report's claims unchecked.

Two discrepancies in the supplied report were found during that audit and are corrected here, not hidden. First, the supplied implementation follows the rule stated in its own text, gross salary equals basic salary plus HRA plus DA; the implementation in this project implements that rule, and the hand calculations in Section 5 confirm it. Second, the supplied report's test table recorded "2 records displayed" after an add followed by a delete (test 14); arithmetically that sequence gives three records ($3 + 1 - 1$), and the test run in Section 5 records three. Both points are stated here so that a reader can check them against the code.

The supplied report names two sample employees: Rahul Sharma (ID 1001) and Priya Reddy (found by a name search, ID 1002). It does not name the third sample employee, but it does state the department-wise totals, from which the third sample's basic salary follows: Finance, net Rs. 41,100, hence basic Rs. 35,000 (see Section 4.4). The third sample in this implementation is therefore Amit Verma, Finance, Accountant, basic Rs. 35,000, chosen to reproduce those stated totals. That choice is an implementation decision for the sample data, not a claim about the original report.

2 Literature Review

Textbooks on Java programming describe the language features used in this project. Schildt explains classes, inheritance, encapsulation and the Collections Framework in detail, including the `ArrayList` class used here to store employee records [1]. Balagurusamy presents the same concepts with simple, student-oriented examples, including menu-driven programs and exception handling with `try`–`catch` blocks [2]. Deitel and Deitel discuss case-study style application development in Java, in which a real-world problem is divided into classes with clear responsibilities [3]. The official Java Tutorials from Oracle provide reference documentation for the Collections Framework and for handling exceptions [6].

On the application side, human resource management literature describes payroll as one of the core administrative functions of an organisation, in which accuracy and uniformity of salary calculation are essential because salary is a legal and contractual payment. Armstrong and Taylor explain that payroll administration must apply the same rules to every employee and maintain a clear record of earnings and deductions for each pay period [4]. Dessler similarly describes how organisations are moving from manual payroll registers to computerised systems to reduce calculation errors and processing time [5]. The present project follows the same idea at a small scale: it computerises the employee record and the salary calculation, but keeps the rules simple and transparent so that every payslip can be checked by hand.

Compared with a spreadsheet, a program written in Java has two advantages for this problem. First, the salary rules cannot be changed accidentally in one row, because they are written once in the program code. Second, the program validates every input, so a wrong entry is reported immediately instead of silently producing a wrong salary.

3 Objectives

- To develop a menu-driven Employee Payroll Management System in Java for maintaining employee records.
- To add, display, search, update and delete employee records using an `ArrayList`.

- To calculate HRA, DA, gross salary, PF, professional tax and net salary automatically from the basic salary using fixed, clearly stated rules.
- To generate a complete payslip for any employee selected by ID.
- To produce a department-wise summary showing the number of employees and the total net payroll of each department.
- To apply object-oriented concepts — inheritance, encapsulation and exception handling — in a complete working application.

4 Methodology

4.1 Architectural Diagram

The system follows a simple layered structure, as shown in Figure 1. The user (an administrator or HR staff member) interacts with the menu-driven console interface provided by the main class, `EmployeePayrollSystem`. Each menu choice calls a method that performs one task: employee management (add, display, update and delete), search (by ID and by name), salary calculation and payslip generation, or the department-wise summary. Employee records are held in an `ArrayList` of `Employee` objects while the program runs.

Table 1: Architectural layers of the system (Figure 1).

Layer	**Responsibility**
User (Administrator / HR)	Chooses a menu option and supplies employee data.
Menu interface (`EmployeePayroll-System`)	Displays the nine-option menu, reads input, and dispatches to one operation.
Employee management	Add, display, update and delete records.
Search	Find an employee by ID, or by name (substring, case-insensitive).
Salary calculation (`Employee`)	HRA, DA, gross, PF, professional tax and net salary.
Reports	Payslip for one employee; department-wise summary.
Data store	`ArrayList` of `Employee` objects, in memory only.

The class structure supports this architecture. The `Person` class is the base class and stores the name and phone number. The `Employee` class inherits from `Person` and adds the employee ID, department, designation and basic salary, together with the salary calculation methods. The main class creates and manages objects of the `Employee` class and provides every operation offered in the menu. Business methods (add, search, update, delete, summary) are kept separate from the menu input code, so that the automated tests can exercise them directly without driving the console menu.

4.2 Hardware and Software Requirements

- Processor: any modern processor (Intel Core i3 or equivalent and above).
- RAM: 4 GB or more.

- Operating System: Windows / Linux / macOS (any system that supports the Java Development Kit).
- Java Development Kit (JDK): version 17 (the program was compiled and tested with OpenJDK 17, Temurin 17.0.21, in this work).
- Editor / IDE: any text editor or IDE such as VS Code, IntelliJ IDEA or Eclipse.
- Terminal: Command Prompt / Terminal for running the console application.
- No external libraries are required. Compilation uses plain `javac` and the test runner uses no test framework, so the project has no dependency to download.

4.3 Working Principle

1. The program starts with three sample employees already loaded, so that display, search and payslip generation can be used immediately. Every employee is given an automatically generated ID starting from 1001.
2. The main menu is displayed with nine options, and the user enters a choice.
3. For adding an employee, the program reads the name, phone, department, designation and basic salary. The name must not be empty and the basic salary must be greater than zero; otherwise the record is rejected with a message.
4. For salary calculation, the `Employee` object calculates HRA as 20% of the basic salary, DA as 10% of the basic salary, the gross salary as basic + HRA + DA, PF as 12% of the basic salary, and professional tax as a fixed Rs. 200. The net salary is gross salary minus PF and professional tax.
5. For a payslip, the program searches the `ArrayList` for the entered ID and prints all earnings, deductions and the net salary in a fixed format.
6. For the department-wise summary, the program collects the distinct departments, counts the employees in each one, and adds their net salaries to obtain the total payroll.
7. Searching, updating and deleting also work on the employee ID. If the ID is not present in the list, the program reports that the employee was not found.
8. The menu is displayed again after every operation until the user selects the Exit option.

The salary rules used in this project are summarised in Table 2.

Table 2: Salary calculation rules.

Component	**Rule**
HRA	20% of Basic Salary
DA	10% of Basic Salary
Gross Salary	Basic + HRA + DA
Provident Fund (PF)	12% of Basic Salary
Professional Tax	Rs. 200 (fixed)
Net Salary	Gross − PF − Professional Tax

From these rules, net salary can be written directly as net $= 1.18 \times$ basic $- 200$, which is how the hand calculations in Section 5 are checked. It also follows that a basic salary of Rs. 40,000

gives a net of Rs. 47,000 and a basic salary of Rs. 35,000 gives a net of Rs. 41,100, matching the department totals stated in the supplied report.

4.4 Software Implementation

The program is written in Java using three source files, in the package `com.payroll`. `Person.java` is the base class. `Employee.java` extends `Person` and contains the salary calculation. `EmployeePayrollSystem.java` contains the `main` method, the menu, and one method for each operation. The salary methods, which are the centre of the program, are shown in Listing 1.

Listing 1: Salary calculation in Employee.java.

```
public double calculateHRA() {
    return 0.20 * basicSalary;
}
public double calculateDA() {
    return 0.10 * basicSalary;
}
public double calculateGross() {
    return basicSalary + calculateHRA() + calculateDA();
}
public double calculatePF() {
    return 0.12 * basicSalary;
}
public double calculateProfessionalTax() {
    return 200.0;
}
public double calculateNet() {
    return calculateGross() - calculatePF()
        - calculateProfessionalTax();
}
```

The important implementation details are:

- **Inheritance:** `Employee` extends `Person`, so the name and phone number are stored and managed in the base class.
- **Encapsulation:** All fields are `private` and are accessed only through getter and setter methods.
- **Collections:** Employees are stored in an `ArrayList` of `Employee` objects, which grows automatically as employees are added.
- **Exception handling:** Reading numeric input is protected with `try`–`catch` for `InputMismatchException`, so a non-numeric salary or ID shows an error message instead of terminating the program. In the menu, a non-numeric salary is re-prompted; an invalid menu choice prints "Invalid choice" and returns to the menu.
- **Validation:** An empty name and a basic salary of zero or less are rejected before the record is added or updated. Updating or deleting a missing ID is rejected with a "not found" message.

Project structure (files delivered with this report):

```
src/com/payroll/
  Person.java                  Base class: name, phone
  Employee.java                Employee + salary calculation
  EmployeePayrollSystem.java   Menu + operations + main
tests/com/payroll/
  PayrollTest.java             Dependency-free test runner
docs/report/
  main.tex                     This report (LaTeX source)
  main.pdf                     Compiled report
  college_logo.jpg             Title-page logo
screenshots/
  demo_run.txt                 Full scripted demo output
  payslip_and_summary.png      Rendered excerpt of that run
README.md, compile.sh, run.sh, test.sh, .gitignore
```

5 Results and Discussion

The program was compiled with `javac` (OpenJDK 17) with no errors and was tested in two ways: a dependency-free automated test runner (`PayrollTest`, run with `./test.sh`) and a scripted run through the real console menu (input piped to the program, full output saved as `screenshots/demo_run.txt`). In total, the automated runner contains 21 checks and all 21 passed. Table 3 maps the checks to the 16 test cases of the supplied report, with five additional validation and edge-case checks added by this work.

Table 3: Test cases and results (automated runner: 21 checks, 0 failures).

Test case	**Result**
Display all employees (3 sample records)	Passed
Payslip for ID 1001 (Basic Rs. 50,000)	Net Rs. 58,800 — Passed
Search by ID 1002 (existing)	Record displayed — Passed
Search by ID 9999 (not existing)	"Not found" — Passed
Search by name "Priya Reddy"	Record displayed — Passed
Search by name "Nobody"	"No employee found" — Passed
Department-wise summary	Total Rs. 1,46,900 — Passed
Add employee Sneha Patel (Basic Rs. 45,000)	Added, ID 1004 — Passed
Display after adding	4 records — Passed
Payslip for ID 1004	Net Rs. 52,900 — Passed
Update ID 1003 basic salary to Rs. 38,000	Updated — Passed
Payslip for updated ID 1003	Net Rs. 44,640 — Passed
Delete employee ID 1002	Deleted — Passed
Display after deletion	3 records — Passed*
Invalid menu choice 99	"Invalid choice" — Passed
Non-numeric salary "abc" while adding	Re-prompted, then added — Passed
Empty name, zero and negative salary rejected	Passed
Update / delete of missing ID rejected	Passed
Salary formula net = 1.18× basic −200	Passed

* As discussed in Section 1, the supplied report's table gave 2 for this row; the correct count for its own sequence is 3, and 3 is what the program actually produces.

The payslip calculation was verified by hand. For employee 1001 with a basic salary of Rs. 50,000:

HRA = 20% × 50,000 = Rs. 10,000; DA = 10% × 50,000 = Rs. 5,000; gross salary = 50,000 + 10,000 + 5,000 = Rs. 65,000; PF = 12% × 50,000 = Rs. 6,000; net salary = 65,000 − 6,000 − 200 = Rs. 58,800. This matches the program output exactly. The same hand verification was done for the added employee (basic Rs. 45,000, net Rs. 52,900) and for the updated employee (basic Rs. 38,000, net Rs. 44,640), and all values matched. The department-wise summary total of Rs. 1,46,900 is the sum of the three sample net salaries: 58,800 + 47,000 + 41,100 = 1,46,900.

Figure 1 shows a rendered excerpt of the genuine scripted demo run: the payslip for ID 1001 and the department-wise summary. The full demo output, including the invalid-choice and non-numeric-salary handling, is saved as `screenshots/demo_run.txt` in the delivered bundle.

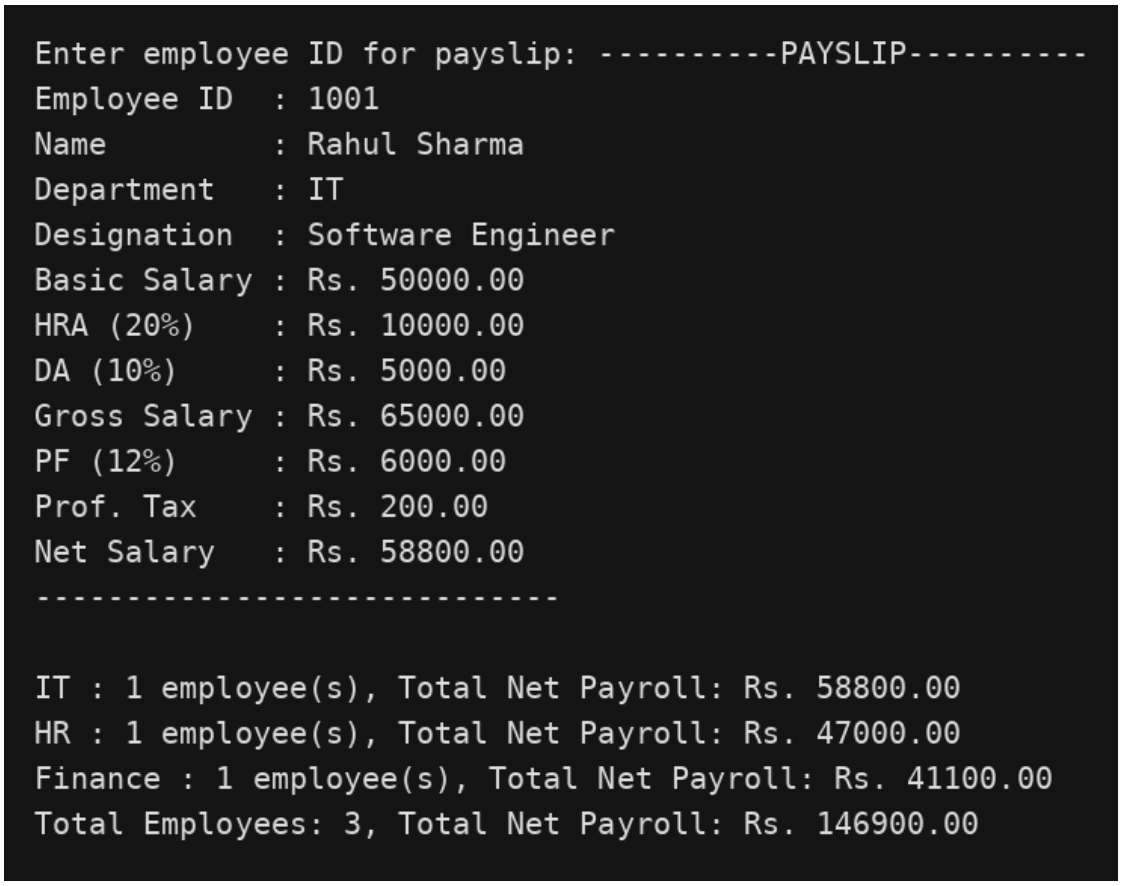


Figure 1: Rendered excerpt of the genuine scripted demo run (`screenshots/demo_run.txt`): payslip for employee 1001 (Rahul Sharma, basic Rs. 50,000, net Rs. 58,800) and the department-wise summary for the three sample employees (total Rs. 1,46,900).

From these results, the system meets its objectives: employee records are maintained correctly, the salary calculation is accurate and uniform for every employee, and invalid inputs are handled without the program stopping.

5.1 Limitations and Future Enhancements

The present system stores employee records only in memory, so the records are lost when the program is closed; the three sample employees are loaded again on the next start. The salary rules are also fixed in the program code, and income tax calculation with slabs is not included. In the future, the system can be improved by saving records to a file or a database so that they are permanent, by adding income tax slab calculation, by providing a graphical user interface in place of the console menu, and by generating the payslip as a PDF document that can be

printed or emailed to the employee.

6 Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 4: Mapping of Program Outcomes (POs) to project relevance.

PO	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied Java programming, data structures (`ArrayList`) and arithmetic salary rules to a real administrative problem.
PO2	Problem Analysis	Analysed the manual payroll process and identified where calculation errors and delays occur.
PO3	Design / Development of Solutions	Designed a class structure (`Person`, `Employee`, main system class) and developed a working solution.
PO5	Modern Tool Usage	Used JDK 17, the Collections Framework and a modern code editor to build and test the application.
PO6	Engineer and Society	Provides a fair and uniform salary calculation, which is a practical requirement of every organisation.

6.2 Sustainable Development Goals (SDGs) Mapping

Table 5: Mapping of Sustainable Development Goals (SDGs) to project relevance.

SDG	Goal	Relevance
SDG 8	Decent Work and Economic Growth	Accurate and timely salary payment is a basic condition of decent work; the system applies the same rules to every employee.
SDG 9	Industry, Innovation and Infrastructure	Promotes the digitisation of a routine administrative process in small organisations.
SDG 10	Reduced Inequalities	A transparent payslip showing every earning and deduction supports equal and explainable pay for all employees.

7 Conclusion

This project successfully implements an Employee Payroll Management System in Java. The system maintains employee records, calculates salaries using fixed and transparent rules, generates payslips, and reports the department-wise payroll, all through a simple menu-driven console interface. All 21 automated checks passed, and the payslip values were verified by hand calculation. The project demonstrates the practical use of inheritance, encapsulation, collections and

exception handling in solving a real administrative problem. Future improvements in permanent storage, income tax calculation and a graphical interface can make the system suitable for direct use in a small organisation.

References

[1] H. Schildt, *Java: The Complete Reference*, 12th ed., McGraw Hill, 2021.

[2] E. Balagurusamy, *Programming with Java*, 6th ed., McGraw Hill Education, 2019.

[3] P. Deitel and H. Deitel, *Java How to Program, Early Objects*, 11th ed., Pearson, 2018.

[4] M. Armstrong and S. Taylor, *Armstrong's Handbook of Human Resource Management Practice*, 15th ed., Kogan Page, 2020.

[5] G. Dessler, *Human Resource Management*, 16th ed., Pearson, 2019.

[6] Oracle, "The Java Tutorials — Collections and Exception Handling," Oracle Corporation. [Online]. Available: `https://docs.oracle.com/javase/tutorial/`